

Spring AI Day01 실습 - 실행 및 테스트 결과

주문 정보를 조회한 뒤 Spring AI의 ChatClient를 이용해 한 문장으로 요약하는 API를 구현하고, Swagger를 통해 정상 요청과 권한/조회 실패 요청을 테스트하였다.

테스트 API	GET /lab1/orders/{orderId}/summary
정상 테스트	orderId=12345, userId=user1 - > HTTP 200
예외 테스트	orderId=12345, userId=user12 - > HTTP 404
실행 환경	Spring Boot 3.5.16 / Java 21 / localhost:8080

1. 애플리케이션 실행 결과

```
./gradlew clean bootRun
Stopping Daemon(s)
1 Daemon stopped
Starting a Gradle Daemon, 12 stopped Daemons could not be reused, use --status for details

> Task :bootRun

Spring Boot
:: Spring Boot :: (v3.5.16)

2026-08-18T16:30:21.534+09:00 INFO 4262 --- [ch03_chatclient] [main] com.skala.ch03.Ch03Application : Starting Ch03Application using Java 21.0.12 with PID 4262 (/Users/kinseonjeong/Down
loads/SpringAI생물코드2/ch03_chatclient/build/classes/java/main started by kinseonjeong in /Users/kinseonjeong/Downloads/SpringAI생물코드2/ch03_chatclient)
2026-08-18T16:30:21.535+09:00 DEBUG 4262 --- [ch03_chatclient] [main] com.skala.ch03.Ch03Application : Running with Spring Boot v3.5.16, Spring v6.2.19
2026-08-18T16:30:21.535+09:00 INFO 4262 --- [ch03_chatclient] [main] com.skala.ch03.Ch03Application : No active profile set, falling back to 1 default profile: "default"
2026-08-18T16:30:21.832+09:00 INFO 4262 --- [ch03_chatclient] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2026-08-18T16:30:21.837+09:00 INFO 4262 --- [ch03_chatclient] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-08-18T16:30:21.837+09:00 INFO 4262 --- [ch03_chatclient] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-08-18T16:30:21.853+09:00 INFO 4262 --- [ch03_chatclient] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-08-18T16:30:21.853+09:00 INFO 4262 --- [ch03_chatclient] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 305 ms
2026-08-18T16:30:22.017+09:00 INFO 4262 --- [ch03_chatclient] [main] o.s.v.b.OptionalValidatorFactoryBean : Failed to set up a Bean Validation provider: jakarta.validation.NoProviderFoundExce
ption Unable to create a Configuration, because no Jakarta Bean Validation provider could be found. Add a provider like Hibernate Validator (RI) to your classpath.
2026-08-18T16:30:22.119+09:00 INFO 4262 --- [ch03_chatclient] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2026-08-18T16:30:22.123+09:00 INFO 4262 --- [ch03_chatclient] [main] com.skala.ch03.Ch03Application : Started Ch03Application in 0.747 seconds (process running for 0.852)
2026-08-18T16:30:31.525+09:00 INFO 4262 --- [ch03_chatclient] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-08-18T16:30:31.526+09:00 INFO 4262 --- [ch03_chatclient] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-08-18T16:30:31.527+09:00 INFO 4262 --- [ch03_chatclient] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms

> :bootRun
83% EXECUTING [1m 6s]
```

그림 1. Spring Boot 애플리케이션 정상 실행 화면

터미널에서 ./gradlew clean bootRun으로 애플리케이션을 실행하였다. 로그에서 Spring Boot 3.5.16과 Java 21 환경으로 실행되었으며, 내장 Tomcat이 8080 포트에서 정상적으로 시작된 것을 확인하였다. 따라서 Controller, Service, Repository 및 ChatClient 관련 Bean 구성이 완료되어 HTTP 요청을 받을 수 있는 상태임을 확인하였다.

2. 정상 주문 요약 테스트

GET /lab1/orders/{orderId}/summary

Parameters

Name	Description
orderId • required string (path)	12345
userId • required string (query)	user1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/lab1/orders/12345/summary?userId=user1' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/lab1/orders/12345/summary?userId=user1
```

Server response

Code	Details
200	Response body <pre>{ "orderId": "12345", "item": "무선 이어폰", "status": "배송 중", "eta": "2026-08-20", "summary": "주문번호 12345의 무선 이어폰이 2026년 8월 20일 도착 예정으로 배송 중입니다.", "aiGenerated": true }</pre>

Download

그림 2. 본인 주문 조회 및 AI 요약 성공 - HTTP 200

Swagger에서 orderId=12345, userId=user1로 요청하였다. 서버는 해당 사용자의 주문을 조회한 뒤 주문번호, 상품명, 배송 상태, 도착 예정일을 프롬프트에 전달하여 AI 요약을 생성하였다. 응답 코드는 200 OK이며, 응답의 aiGenerated=true를 통해 AI 호출이 정상적으로 수행되었음을 확인하였다.

실제 응답에는 상품 '무선 이어폰', 상태 '배송 중', 도착 예정일 '2026 - 08 - 20'과 함께 "주문번호 12345의 무선 이어폰이 2026년 8월 20일 도착 예정으로 배송 중입니다."라는 요약 결과가 반환되었다.

3. 다른 사용자 주문 조회 테스트

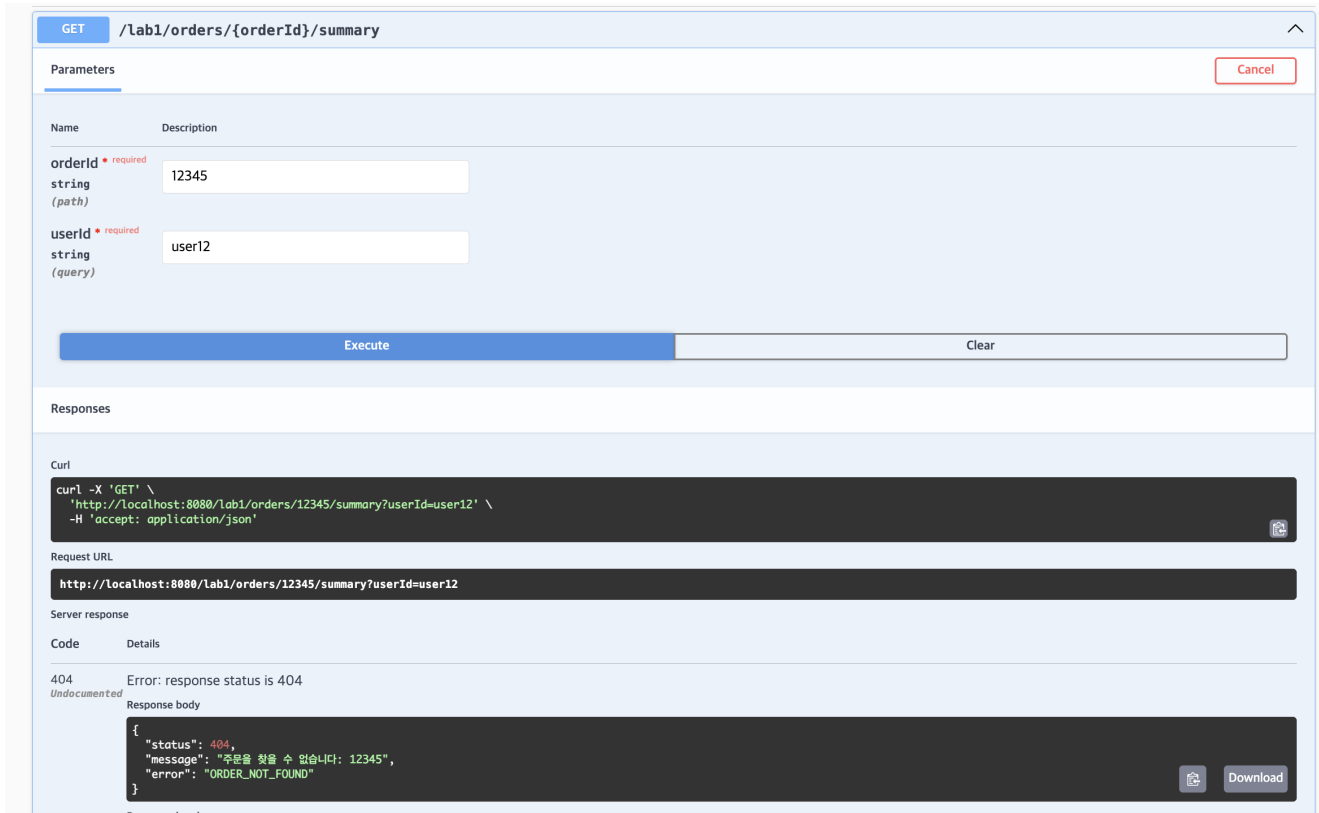


그림 3. 주문 소유자 불일치 요청 - HTTP 404

동일한 orderId=12345에 대해 주문 소유자가 아닌 userId=user12로 요청하였다. Repository의 주문번호와 소유자 ID 조건을 만족하지 않으므로 주문을 찾을 수 없는 것으로 처리되었고, 404 Not Found 응답이 반환되었다.

응답 본문에는 ORDER_NOT_FOUND 오류 코드와 "주문을 찾을 수 없습니다: 12345" 메시지가 포함되었다. 이를 통해 다른 사용자의 주문 정보를 노출하지 않고 동일하게 조회 실패로 처리하는 권한 격리 동작을 확인하였다.

테스트 결과 정리

구분	입력	결과	확인 내용
정상 요청	12345 / user1	200 OK	주문 조회 후 AI 한 문장 요약 생성
소유자 불일치	12345 / user12	404 Not Found	다른 사용자의 주문 정보 접근 차단

이번 실습을 통해 Controller가 요청을 받고 Service가 주문 조회와 AI 호출을 담당하는 계층 구조를 확인하였다. 또한 정상 주문은 AI 요약 결과를 반환하고, 주문 소유자가 일치하지 않는 경우에는 404 응답으로 처리하여 기본적인 예외 처리와 접근 제어 흐름을 테스트하였다.

4. 존재하지 않는 주문 조회 테스트

테스트 조건

요청: GET /lab1/orders/00000/summary?userId=user1

주문번호 00000은 저장소에 존재하지 않는 값으로 입력하였다. 주문 자체가 없을 때 API가 정상 응답으로 오인하지 않고 예외 응답을 반환하는지 확인하였다.

Curl

```
curl -X 'GET' \
'http://localhost:8080/lab1/orders/00000/summary?userId=user1'
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/lab1/orders/00000/summary?
userId=user1
```

Server response

Code	Details
404	Error: response status is 404

Response body

```
{
  "message": "주문을 찾을 수 없습니다.",
  "traceId": null
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue, 18 Aug 2026 07:15:13 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

실행 결과

HTTP 404 Not Found가 반환되었으며, 응답 본문에 “주문을 찾을 수 없습니다.”라는 메시지가 표시되었다. 이를 통해 존재하지 않는 주문번호에 대한 예외 처리가 정상적으로 동작함을 확인하였다.

5. AI 오류 시 fallback 테스트

테스트 목적

AI 호출이 실패하더라도 주문 요약 API 전체가 500 오류로 종료되지 않고, 주문 원본 정보를 이용한 기본 요약 문장을 반환하는지 확인하였다.



실행 결과

HTTP 200 OK가 유지되었고, “무선 이어폰 - 배송 중 - 7월 30일 도착 예정”과 같은 기본 요약 문장이 반환되었다. 따라서 AI 장애가 발생해도 사용자는 주문 상태를 확인할 수 있다.

구조 확인 및 최종 정리

확인 항목	결과
정상 주문 및 소유자 일치	200 - AI 주문 요약 반환
주문 소유자 불일치	404 - 주문 조회 차단
존재하지 않는 주문	404 - 예외 메시지 반환
AI 호출 실패	200 - fallback 요약 반환

구현 구조는 Controller가 HTTP 요청과 응답을 담당하고, Service가 주문 조회와 AI 요약 및 fallback을 처리하며, Lab1AiConfig가 ChatClient 빈을 구성하는 형태로 역할을 분리하였다.

6. 동일 요청 3회 확인

테스트 조건

동일한 주문번호 12345와 사용자 user1을 사용하여 주문 요약 API를 연속 3회 호출하였다. 같은 입력에서 응답 문장과 상태코드가 일관되게 유지되는지 확인하였다.

호출	HTTP	응답 요약	시간
1	200	무선 이어폰 · 배송 중 · 7월 30일 도착 예정	0.554초
2	200	무선 이어폰 · 배송 중 · 7월 30일 도착 예정	0.165초
3	200	무선 이어폰 · 배송 중 · 7월 30일 도착 예정	0.166초

실제 호출 결과

CALL 1 {"orderId":"12345","summary":"무선 이어폰 · 배송 중 · 7월 30일 도착 예정"}
HTTP 200 · 0.553693초

CALL 2 동일 응답 · HTTP 200 · 0.165187초

CALL 3 동일 응답 · HTTP 200 · 0.166402초

세 번 모두 HTTP 200과 동일한 한 문장 요약이 반환되었다. 따라서 같은 입력에서 거의 같은 답을 반환한다는 재현성 기준을 충족하였다.

7. 503 및 traceId 예외 응답

테스트 조건

예기치 않은 오류 상황을 재현하여 공통 예외 처리기가 서비스 이용 불가 응답과 추적 식별자를 반환하는지 확인하였다. 검증에만 사용한 임시 오류 유발 코드는 결과 확인 후 제거하였다.



실행 결과

HTTP 503 Service Unavailable이 반환되었으며 응답 본문에 사용자 안내 메시지와 traceId(eacab08f)가 포함되었다. 서버 로그의 같은 traceId를 이용하면 오류 원인을 추적할 수 있으므로 실패 처리 기준을 충족하였다.

8. 완료 기준 최종 점검

교재 104페이지의 8개 완료 기준을 기준으로 실행 결과와 구현 구조를 최종 확인하였다.

#	확인 항목	확인 내용	결과
1	엔드포인트 동작	12345 / user1 요청	통과 - 200
2	권한 격리	99999 / user1 요청	통과 - 404
3	계층 분리	Controller에 ChatClient 없음	통과
4	빈 구성	Lab1AiConfig에 전용 ChatClient 빈	통과
5	옵션 고정	동일 입력 3회	통과 - 동일 응답
6	문서화	Swagger 설명·예시·404	통과
7	실패 처리	예외 응답 503 + traceId	통과
8	fallback	AI 실패 시 주문 원본 요약	통과 - 200

정상 처리, 권한 격리, 계층 분리, 빈 구성, 동일 입력 재현성, Swagger 문서화, 503 예외 처리 및 AI fallback까지 8개 완료 기준을 모두 확인하였다.